AFTERMATH AI

Hackathon Ideas & Practical Build Blueprint

WeMakeDevs Hack · Best Business Workflow Track

Core concept
An evidence-based recovery layer for business workflows that fail halfway across connected systems. GhostTrail investigates what happened; Aftermath plans, safely executes and verifies recovery.

This document consolidates the three shortlisted concepts—Aftermath AI, ChangeShield and GhostTrail—then develops Aftermath + GhostTrail into a practical, judge-focused MVP plan.

What is inside

- Three serious, differentiated product concepts and a comparison.
- Recommended product scope, problem statement and cautious novelty positioning.
- A reproducible e-commerce incident demo with architecture, agent roles and safety rules.
- Data model, state machine, idempotency, compensation and human approval.
- MVP scope, 24-hour implementation plan, pitch, demo script and evaluation metrics.
- Competitor and technical references to verify before presenting claims.

Design principle: show a real failure, traceable evidence, a safe decision, and a verified outcome—not agents merely chatting.

1. Executive recommendation

Build **Aftermath AI** as the main product and use **GhostTrail** as its investigation engine. Keep **ChangeShield** as a possible later extension. This creates one cohesive workflow instead of three disconnected demos.

Concept	Core job	Why it matters	Best demo
Aftermath AI	Recover partially failed workflows safely	Avoids manual investigation, inconsistent records and unsafe retries	Paid order; shipping times out; reconcile and recover
ChangeShield	Predict the impact of a proposed change	Helps prevent outages and costly downstream surprises	Find a hidden dependency before API deprecation
GhostTrail	Reconstruct what happened across disconnected tools	Cuts time spent correlating conflicting records	Explain why onboarding appears complete but access fails

Why Aftermath + GhostTrail is the strongest combination

- **Clear business pain:** a workflow can partially succeed and leave payment, inventory, order and shipping records inconsistent.
- **Demonstrable technical depth:** event correlation, state reconciliation, idempotency, compensation, approval and read-back verification.
- **Observable outcome:** judges can see the initial failure, evidence, recovery decision and final consistent state.
- **Feasible scope:** a controlled mock environment avoids live-payment risk and makes the demo repeatable.

The novelty claim should be the proposed combination and implementation—not a claim that workflow recovery, incident investigation or human approval is unprecedented.

2. The three shortlisted ideas

2.1 Aftermath AI — Self-recovering business workflows

Problem: workflows spanning order management, payments, inventory and shipping can fail halfway. A blind retry may duplicate side effects; a blind rollback may create a new problem.

How it works: an investigator reconstructs events; a state reconciler checks current records; a recovery planner compares retry, compensation and escalation; a policy layer enforces safety; a verifier checks the final state.

Example: payment succeeds and inventory is reserved, but shipping creation times out. Aftermath checks whether a shipment exists before retrying and avoids charging the customer again.

Unique angle: focuses on the actual business state and safe recovery, not just detecting errors or generating incident summaries.

Stack: Discord/Slack, TypeScript, PostgreSQL, durable workflow engine, mock SaaS APIs and deterministic policy checks.

2.2 ChangeShield — AI pre-mortem for business changes

Problem: a change that looks harmless in one system may violate commitments or break downstream workflows.

How it works: dependency analysis traces services and workflows; evidence agents check repository references, logs and configuration; business-impact analysis maps affected customers and deadlines; an adversarial reviewer challenges gaps; the system produces a go/no-go recommendation with mitigations.

Example: before deprecating an API endpoint, ChangeShield finds a hidden service dependency and blocks approval until a migration is planned.

Unique angle: evidence-backed impact mapping before execution, distinguishing confirmed dependencies from plausible risks.

2.3 GhostTrail — Cross-platform business forensic agent

Problem: CRM, support, identity and payment tools may disagree about what happened, forcing teams to manually reconstruct timelines.

How it works: a timeline agent correlates events; a contradiction agent flags conflicts; a causal investigator identifies the earliest supported failure; an evidence auditor links claims to records; a remediation agent proposes the smallest corrective action.

Example: the CRM says an employee is onboarded, but the employee cannot access the product. GhostTrail traces the failed identity-provider step and links the evidence.

Unique angle: separates confirmed facts, inferences and unknowns rather than merely summarizing logs.

3. Comparison and product differentiation

Criterion	Aftermath AI	ChangeShield	GhostTrail
Primary value	Recover failed operations	Prevent risky changes	Explain failures
Executes actions	Yes, with safeguards	Usually approval / mitigation workflow	Optional remediation
Demo impact	Very high: visible recovery	High: visible hidden dependency	High: solve a cross-system mystery
Main challenge	Safe recovery logic	Reliable impact coverage	Correct event correlation
Recommended role	Main product	Future extension	Investigation engine

Differentiated value proposition

Aftermath is an evidence-backed recovery layer: it reconstructs the event history, reconciles current system state, proposes a constrained recovery plan, requests approval for risky actions and verifies the result.

Potential differentiators to demonstrate

1. **Cross-system evidence reconciliation:** tell the difference between an API timeout, a failed operation and a successful operation whose response was lost.
2. **Recovery selection:** compare safe continuation, retry, compensation and escalation against explicit business invariants.
3. **Verified closure:** do not mark the incident resolved just because an API returned success; read back affected records and validate consistency.

These are hypotheses for product positioning, not proof of an unoccupied market. Existing products overlap with individual parts of the design.

Competitor and novelty check

- **incident.io:** a reference point for AI incident investigation, telemetry correlation and response workflows.
- **Temporal:** a reference for durable workflows and Saga-style compensating actions.
- **Inngest:** a reference for durable execution, retries and human-in-the-loop workflows.
- **Switch AI:** relevant to connecting agents and people in chat-based workflows.

Before pitching novelty, spend 30–45 minutes checking official product pages and docs. Search for “AI workflow recovery”, “agentic transaction repair”, “distributed transaction reconciliation” and “AI business process exception handling”. Compare investigation, state reconciliation, recovery planning and verified execution. Record only capabilities supported by public documentation or demos.

4. Problem statement and target users

Problem statement: Business workflows span independent services. When one step fails after earlier steps have succeeded, the company can be left with incomplete operations and conflicting records. Teams must manually inspect several tools, determine the actual state and choose a safe recovery action.

Target users:

- E-commerce engineering and site reliability teams.
- Operations teams managing orders, payments and fulfilment exceptions.
- SaaS companies running integrations across several APIs.

- Developers responsible for business automations.

Initial scope: one order-processing workflow, four simulated services, one chat interface and a small set of explicitly permitted recovery actions.

5. End-to-end demo: a paid order that cannot ship

Use deterministic synthetic data so the scenario can be reset and replayed. Example order: **ORD-1042**, amount ■**4,999**.

Step	System event / state	What the system should infer
1	Order service: CONFIRMED	Order exists
2	Payment service: CAPTURED	Payment succeeded, subject to current authoritative read-back
3	Inventory service: RESERVED	Inventory is reserved
4	Shipping create request times out	Shipment status is unknown; timeout is not proof of failure
5	A PENDING payment webhook arrives later	There is a conflict; event ordering and authoritative state must be checked

Expected Aftermath behavior

1. **Detect:** collect the shipping timeout and conflicting payment event.
2. **Reconstruct:** GhostTrail builds a timestamped timeline with source and correlation ID for each event.
3. **Reconcile:** query payment, order, inventory and shipping records for their current state.
4. **Classify:** label facts as confirmed, stale, inferred or unknown. Determine whether a shipment may already exist.
5. **Propose:** if payment and inventory are confirmed and no shipment exists, propose a shipment retry with the original idempotency key. If shipment state is unknown, reconcile before creating anything.
6. **Approve:** show action, evidence, expected effects and risks in Discord or Slack.
7. **Execute and verify:** perform the permitted action, read back the resulting records and mark recovered only when invariants hold.

Critical safety rule: never refund a captured payment or retry a potentially non-idempotent operation merely because an LLM recommends it. Ambiguous external state requires reconciliation or escalation.

6. Architecture and data flow



Recommended stack

Layer	MVP choice	Reason
Application	TypeScript + Node.js	One language across bot, APIs and workflow
Dashboard	Next.js + Tailwind	Timeline, evidence, approvals and status
Chat	Discord bot OR Slack app	Choose one for the demo
Workflow engine	Inngest	Durable steps, retries and approval waits
Database	PostgreSQL	Orders, events, evidence, actions and audit
Validation	Zod + TypeScript rules	Schema validation and deterministic controls
AI	Tool-calling LLM with structured outputs	Investigation and recovery proposals
Test services	Small Express APIs or equivalent	Controllable states and failure injection

Do not build a large agent framework from scratch. Four narrow roles plus deterministic service functions are enough for the MVP.

7. Agent roles and deterministic controls

Component	Responsibility	Must not do
GhostTrail investigator	Correlate events; produce evidence-linked timeline	Invent missing events or present assumptions as facts
State reconciler	Read current records; classify conflicts	Assume a timeout means failure
Recovery planner	Compare retry, compensation and escalation	Execute its own proposal
Verification agent	Independently assess whether a plan addresses the failure	Override policy or approve itself
Policy engine	Enforce permissions, state transitions and invariants	Delegate the safety decision to an LLM

The first four roles can share one LLM with different structured prompts if time or compute is limited. Keep the policy engine as ordinary code.

Deterministic safety controls

- Allowlist executable actions and validate all parameters.
- Check explicit preconditions before each mutation.
- Use idempotency keys for supported external operations.
- Require approval for refunds, cancellations, inventory release and other high-impact actions.
- Limit retries and investigation loops.
- Audit proposals, approvals, execution and verification.
- Fail closed if state is ambiguous or verification is unavailable.

Example structured agent output

```
{
  "finding": "Shipping status is unknown",
  "evidence_ids": ["evt_04", "readback_ship_02"],
  "confidence": 0.91,
  "unknowns": ["Whether a shipment exists remotely"],
  "recommended_next_step": "reconcile_shipping",
  "side_effects": [],
  "requires_approval": false
}
```

Confidence is an agent estimate, not a calibrated probability. Evidence IDs must resolve to real event-store records; reject findings that reference nonexistent evidence.

8. Data model and event schema

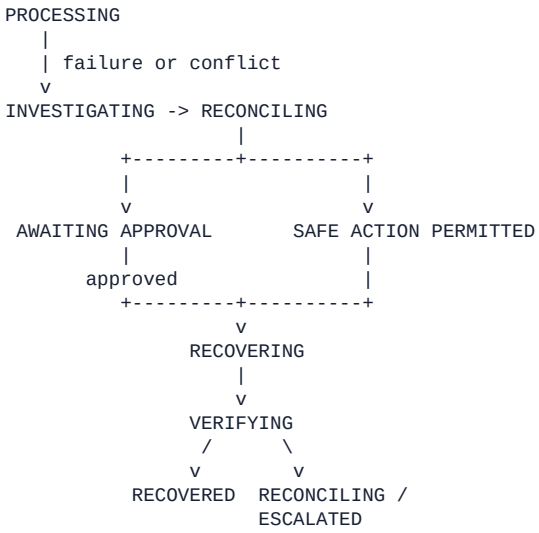
Table	Essential fields
orders	order_id, amount, currency, status, version
events	event_id, order_id, source, event_type, occurred_at, received_at, correlation_id, payload, payload_hash
evidence	evidence_id, order_id, source_event_ids, finding, confidence, unknowns
recovery_plans	plan_id, order_id, steps, preconditions, risk_level, status
actions	action_id, plan_id, idempotency_key, approval_status, execution_status, result
audit_log	actor, action, timestamp, decision, reason

Example event

```
{
  "event_id": "evt_004",
  "order_id": "ORD-1042",
  "source": "shipping",
  "event_type": "shipment.create_timeout",
  "occurred_at": "2026-10-28T14:31:00Z",
  "received_at": "2026-10-28T14:31:08Z",
  "correlation_id": "wf_1042",
  "operation_id": "ship_1042",
  "payload": {
    "request_status": "timeout",
    "remote_status": "unknown"
  }
}
```

Store both event occurrence time and receipt time. A delayed webhook must not automatically overwrite a newer verified state. Preserve original events rather than editing history.

9. State machine, invariants and recovery



Required business invariants

1. At most one shipment is created per order in the mock provider.
2. An order cannot be marked shipped until a shipment record is verified.
3. An ambiguous payment state cannot trigger a second charge or automatic refund.
4. Inventory cannot be released while an active shipment or unresolved fulfilment operation could depend on it.
5. The workflow cannot be marked recovered while required records disagree.

Idempotency and compensation

- An idempotency key identifies a business operation, not an attempt. Example: `ship:ORD-1042:v1` remains the same across retries.
- If a shipment exists, reuse its ID; do not create another.
- If shipment status is unknown, reconcile before retrying.
- If payment is confirmed captured, do not charge again.
- Refunds or cancellations require an approved, state-checked compensating action.
- If compensation fails, preserve the partial state and escalate; do not claim rollback succeeded.

Compensation is not always a literal reversal. A refund cannot erase the fact that a payment was previously captured.

10. MVP versus stretch goals

MVP — must build	Stretch — only if time remains
One Discord or Slack interface	Both Slack and Discord
Four mock services	Second scenario such as employee onboarding
Evidence-linked timeline	Event replay and history comparison
One seeded failure + conflicting event	Compare recovery strategies and estimated costs
Human approval + policy checks	Business-specific configurable policies
Idempotent shipment creation + verification	Real sandbox integration after mock reliability
Small before/after dashboard	More workflow templates

11. 24-hour implementation plan

Time	Deliverable
Hours 0–2	Define states, invariants, event schema and demo script
Hours 2–6	Build mock APIs, seed data and deterministic failure injection
Hours 6–9	Implement event collection, PostgreSQL storage and timeline
Hours 9–13	Implement GhostTrail investigation and structured output
Hours 13–16	Build recovery planner, policy checks, approval and idempotency
Hours 16–19	Add read-back verification, dashboard and chat integration
Hours 19–22	Run failure tests, improve error handling, polish demo
Hours 22–24	Freeze features, rehearse pitch, prepare recorded fallback

Parallel work

- Person A: mock services, state machine and invariants.
- Person B: investigation, evidence ledger and recovery planning.
- Person C: chat integration, dashboard and demo polish.

If the team is smaller, finish the workflow and tests before polishing the UI.

12. Judge-facing pitch

> *“Business workflows rarely fail cleanly. A payment may succeed while shipping fails, a timeout may hide a successful operation, and different systems may disagree about what happened.*
>
> *Aftermath AI is an evidence-based recovery layer for these failures. GhostTrail reconstructs the event history across connected services, the recovery planner proposes the safest next action, and a deterministic policy engine enforces safety rules before anything executes.*
>
> *Instead of blindly retrying, refunding or escalating every failure to a human, Aftermath reconciles the actual state, requests approval for risky actions and verifies the final outcome.*
>
> *We're demonstrating this on a deliberately broken e-commerce order: payment succeeds, inventory is reserved, shipping times out, and payment events conflict. Aftermath must establish the facts, avoid duplicate side effects and recover the order safely.*
>
> *Our goal is not to replace business operators. It is to give them an auditable, controlled way to recover workflows that would otherwise require manual investigation.”*

13. Five-minute demo script

Time	What to show
0:00–0:40	Trigger the order scenario and show inconsistent records.
0:40–1:40	Show GhostTrail timeline, source evidence, stale event and unknown shipping status.
1:40–2:40	Show the proposed action, preconditions, side effects and why blind retry/refund is unsafe.
2:40–3:30	Approve the safe recovery action from the chat interface.
3:30–4:30	Show shipment ID, unchanged payment record, inventory consistency and recovered order.
4:30–5:00	Replay the operation and prove it does not create a duplicate shipment.

The replay is important: it demonstrates a concrete safety property rather than just a persuasive AI explanation.

14. Evaluation metrics

Metric	Measurement
Root-cause accuracy	Correct supported cause / scenarios with a known cause
Evidence coverage	Findings backed by valid records / all factual findings
Safe recovery rate	Scenarios reaching a verified valid state / recoverable scenarios
Unsafe action rate	Invariant-breaking actions / all executed actions
Duplicate side effects	Count of duplicate charges or shipments
Verification accuracy	Correct recovered vs unresolved classifications
Time to recovery	Time from injected failure to verified resolution
Human intervention rate	Scenarios requiring manual resolution / total scenarios

Test plan

Create 20–30 seeded scenarios: normal failures, stale events, lost responses, duplicate events, ambiguous states and recovery failures. Aim for zero duplicate charges and zero invariant-breaking actions in the test suite. Any such action should fail the test regardless of average accuracy. Thirty successful cases do not establish production reliability or justify a 99% claim.

15. Failure cases and security boundaries

Failure	Required behavior
Payment provider timeout	Read current state; don't blindly charge again
Shipment exists but response was lost	Find and reuse existing shipment
Old webhook arrives late	Preserve it as evidence; don't overwrite state blindly
Invalid LLM output	Reject and retry within a limit, or escalate
Evidence sources disagree	Explain conflict and preserve uncertainty
Approval rejected	Cancel proposed action; retain audit trail
Database or verification unavailable	Stop mutations and report verification incomplete
Recovery partially succeeds	Record completed effects and reconcile before next action

Security checklist

- Use synthetic customer and payment data.
- Keep API keys in environment variables or a secret manager, never in prompts or logs.
- Authenticate chat commands and restrict approval to authorized demo users.
- Treat messages, event payloads and repository content as untrusted data, not instructions.
- Use least-privilege service credentials and a fixed action allowlist.
- Record approver, plan version, execution result and timestamps.
- Do not expose raw payment details or sensitive customer data in the dashboard.

- Keep all external side effects inside simulated services during the hackathon.

16. Useful references and verification notes

These references are starting points for implementation and competitor research. Verify current feature details and terms on official pages before making claims in the pitch.

Inngest — durable execution: https://www.inngest.com/docs/durable-execution

Temporal — design patterns: https://docs.temporal.io/design-patterns

Stripe — idempotency: https://stripe.com/blog/idempotency

Switch AI — getting started: https://www.switchagents.ai/get-started

incident.io — AI incident investigation: https://incident.io/solution/ai-sre

Graphite — AI code review guide (original idea reference): https://graphite.com/guides/best-ai-model-for-code-review

Splunk Observability (original ShipOps inspiration): https://www.splunk.com/en_us/blog/conf-splunklive/splunk-observability-announces-new-capabilities-to-help-you-build-troubleshoot-and-innovate-faster.html

Final recommendation

Build Aftermath as a recovery system with GhostTrail as its investigation engine. The strongest judge-facing evidence is a reproducible partial failure, a traceable timeline, a safe decision, an approval-controlled recovery and automated proof that the final state is consistent and retries do not duplicate side effects.

Position the novelty as the specific integration of evidence reconciliation, constrained recovery planning and verified execution, subject to further competitor research. Do not claim guaranteed uniqueness, production-grade reliability or unmeasured business impact.